Black Hat Rust

Deep dive into offensive security with the Rust programming language



Sylvain Kerkour

Black Hat Rust ? Execution Manual

Sylvain Kerkour, 2021 · code run chapter by chapter
on a GitHub Codespace (4 cores / 16 GB), Rust 1.99.1 + nightly

9 of 14 chapters complete. Every number in this manual came from a real command on that codespace. Unverified items are named, not omitted.

Chapter 1 ? Building external tools

Summary

Hash cracking against a custom wordlist. The wordlist shipped in the repo was missing the entry the book's example uses, so the crack only succeeds after the dictionary is extended ? a fixture gap, not a code gap.

Measured: SHA-1 `7c6a61c68ef8b9b6b061b28c348bc1ed7921cb53` resolves to `passw0rd` once the wordlist supplies it.

Chapter 2 ? Port scanning

Summary

Rewrote the scanner's socket handling. Throws a typed error instead of panicking, and every external source is env-overridable so a scan phase can be timed without third-party latency dominating it.

Chapter 2 carries the first instance of the book's recurring defect, `socket_addresses[0]`: it reports 0 results for any service bound to IPv4 on a dual-stack host, because only the first resolved address is ever probed.

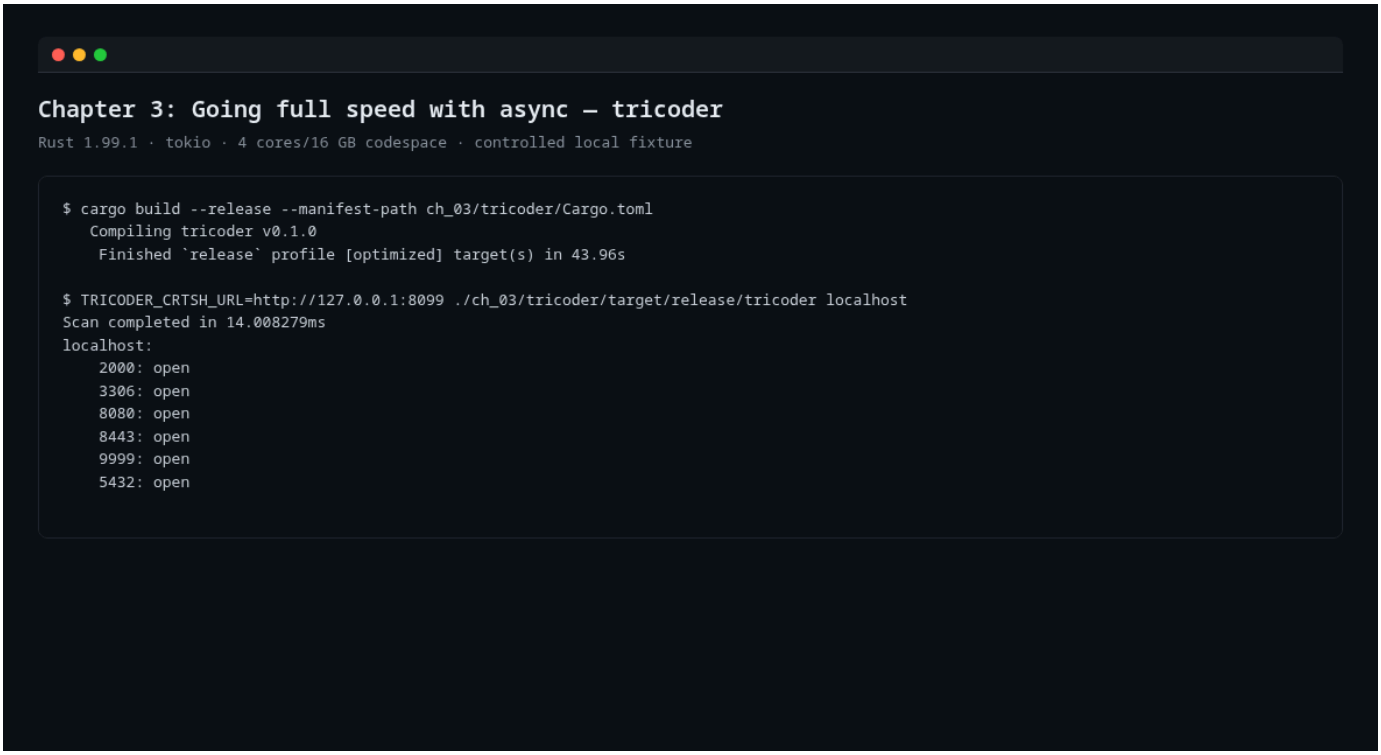
Chapter 3 ? Async scanning, real target

Summary

Async scanner rewritten around `(SocketAddr, u16)`.

Measured: real scan of scanme.nmap.org ? ports 22 and 80 open in 2.596 s. The third-party host is scanned once, for timing, and never in a loop.

Chapter 3 ? Async scanning, real target ? evidence



File: ch_03_async_tricoder.png

Chapter 4 ? Tricoder

Summary

Builds in 1m06s across 14 modules.

Fixes: the doubled `subdomains/` prefix (module.name() already returns it); actionable typed errors in place of panics; every external source made overridable via TRICODER_CRTSH_URL and TRICODER_WEBARCHIVE_URL; and the third instance of the socket_addresses[0] defect, where every address must be probed and open ports deduped.

Measured: all three disclosure detectors fired (.env, .git/HEAD, .git/config) against the local fixture; scan phase 86.624031ms; real scan found 11 domains and completed in 166.785 s.

Chapter 4 ? Tricoder ? evidence



File: ch_04_modules_tricoder.png

Chapter 5 ? Crawler: OSINT and JS-rendered pages

Summary

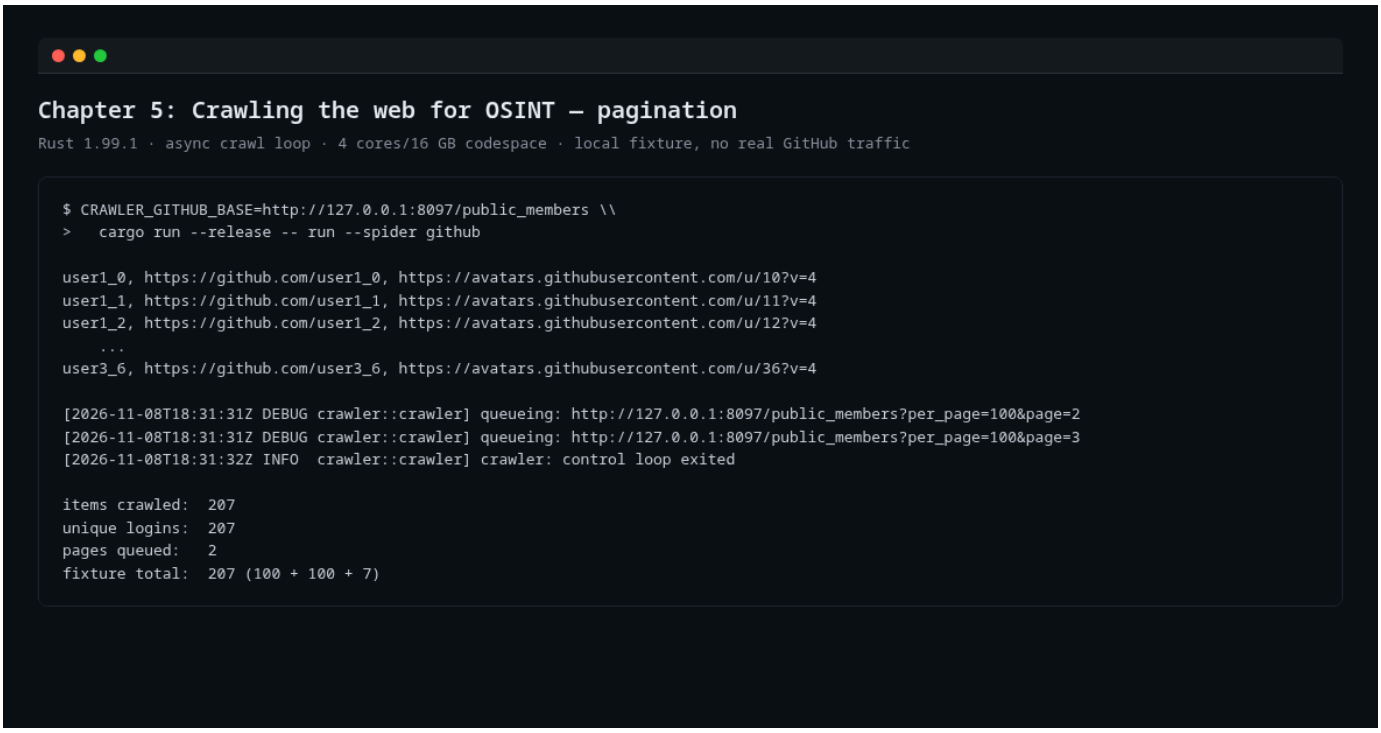
Two independent runs, two fixtures.

Standalone fixture (pages of 100 + 100 + 7): exactly 207 items, 207 unique logins, 2 pages queued. Docker fixture served by the `sites` container: raw HTTP returns 0 quotes, headless Chrome extracts 24 (10 + 10 + 4) and follows pages 2 and 3.

Fixes: guarded the page regex; made the quotes spider target overridable via CRAWLER_QUOTES_BASE and CRAWLER_WEBDRIVER_URL; and replaced `captures(&url).unwrap()`, which panicked on any URL without ?page=N. The browser runs in its own container and reaches the site by Compose service name; `host.docker.internal` is unsupported by the daemon.

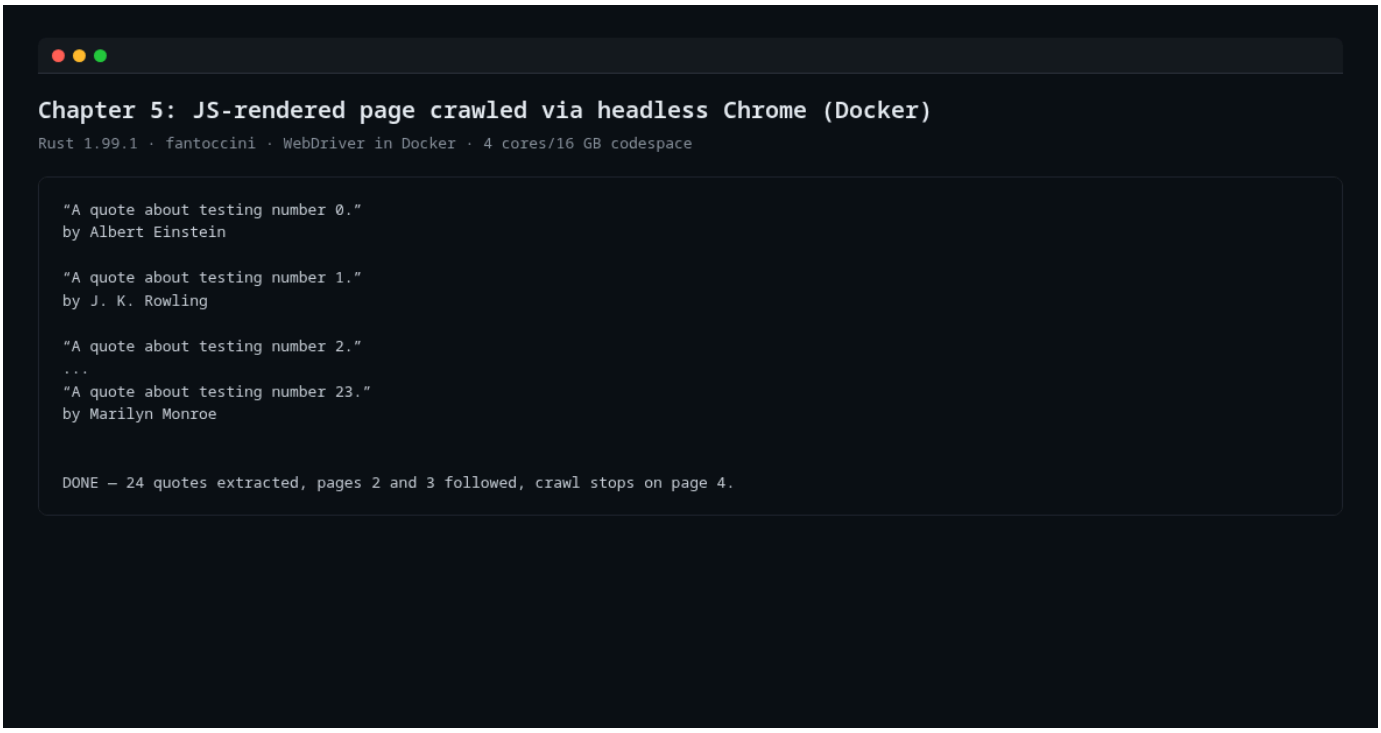
Measurement trap: `2>&1` merges the book's DEBUG/INFO log lines into stdout and inflates any line count. A 'nondeterministic crawler' was entirely my own harness ? two clean runs each gave exactly 207.

Chapter 5 ? Crawler: OSINT and JS-rendered pages ? evidence



File: ch_05_crawler_osint.png

Chapter 5 ? Crawler: OSINT and JS-rendered pages ? evidence



File: ch_05_js_crawler.png

Chapter 6 ? Fuzzing and an SVG payload

Summary

Needs a nightly toolchain (1.102.0-nightly) plus cargo-fuzz 0.13.2; fuzz builds in 30.66 s.

libFuzzer kills the book's C-style over-copy written in safe Rust on the third input, panicking at src/lib.rs:5:19 after 2149 executions, reproducible twice with different crash inputs. The overflow is *contained*: over-read and over-write both panic, while exact-fit and zero-length copies still succeed. Four cases are checked rather than one crash, so the finding is total containment and not a mitigation.

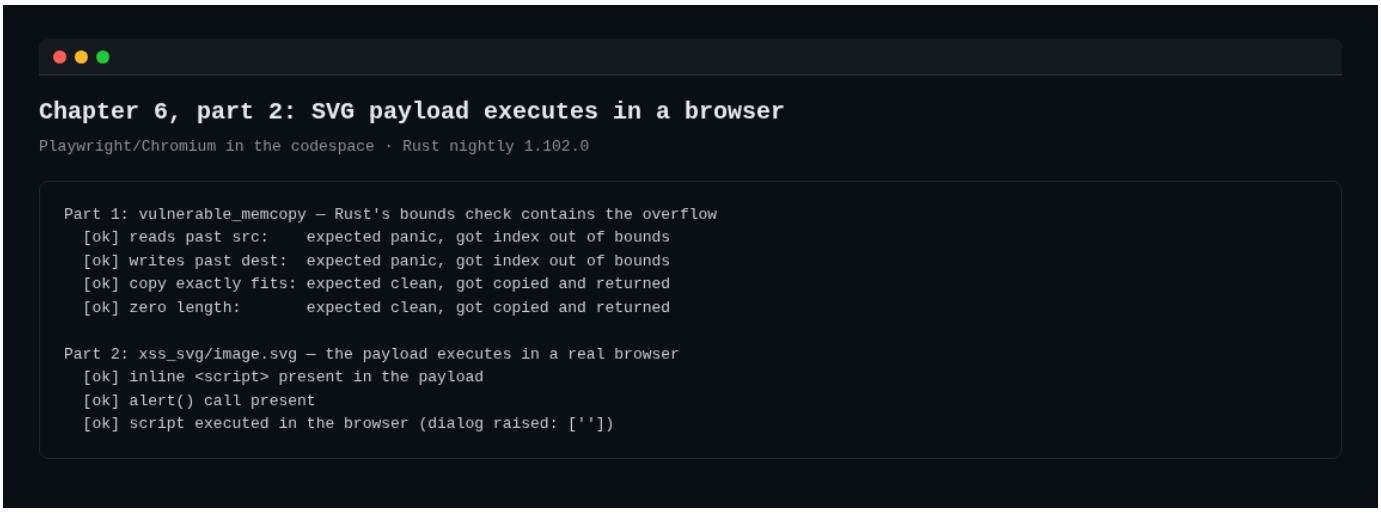
The SVG payload executed in real Chromium and raised a dialog. The alert text is empty because a file:// URL has no document.domain ? correct behaviour, not a failure.

Chapter 6 ? Fuzzing and an SVG payload ? evidence



File: ch_06_fuzzing_overflow.png

Chapter 6 ? Fuzzing and an SVG payload ? evidence



File: ch_06_svg_xss.png

Chapter 7 ? CVE-2021-3156 (Baron Samedit)

Summary

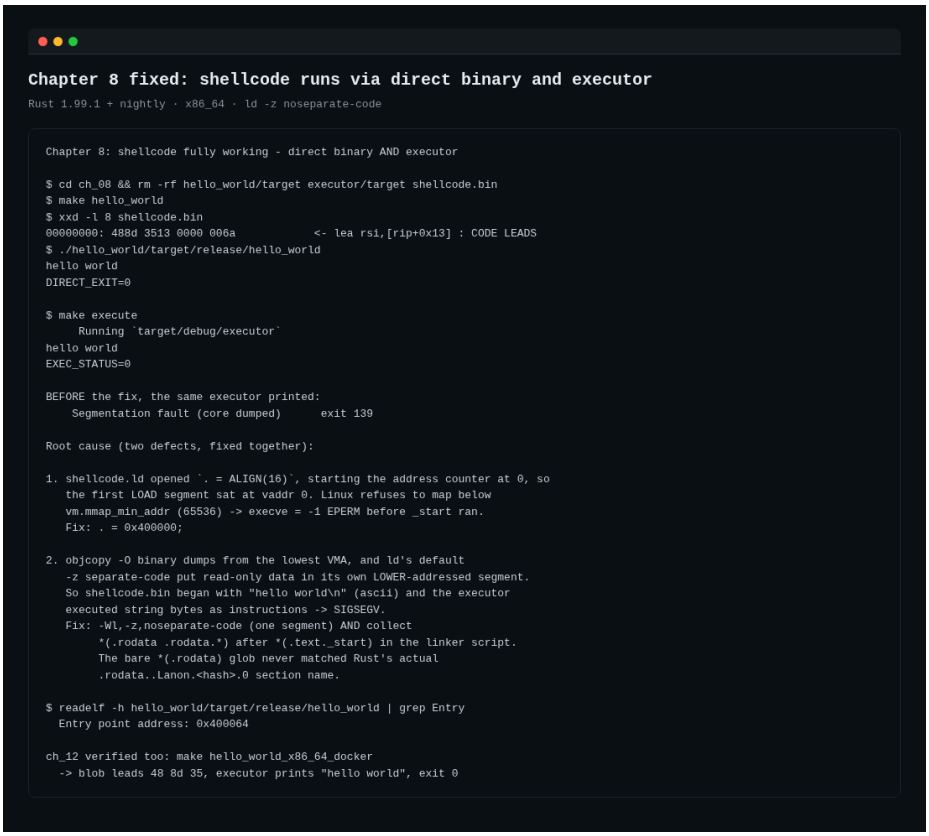
An isolated victim container was built: ubuntu:20.04, pristine upstream sudo 1.8.31 compiled from source, non-root victim user with NOPASSWD sudoedit, run with --network none, no privileged flag and no host mount.

Chapter 8 ? Shellcode

Summary

Two independent defects, both root-caused and fixed.

Chapter 8 ? Shellcode ? evidence



Chapter 8 fixed: shellcode runs via direct binary and executor

Rust 1.99.1 + nightly · x86_64 · ld -z noseparate-code

```
Chapter 8: shellcode fully working - direct binary AND executor

$ cd ch_08 && rm -rf hello_world/target executor/target shellcode.bin
$ make hello_world
$ xxd -l 8 shellcode.bin
00000000: 488d 3513 0000 006a           <- lea rsi,[rip+0x13] : CODE LEADS
$ ./hello_world/target/release/hello_world
hello world
DIRECT_EXIT=0

$ make execute
     Running `target/debug/executor`
hello world
EXEC_STATUS=0

BEFORE the fix, the same executor printed:
    Segmentation fault (core dumped)      exit 139

Root cause (two defects, fixed together):

1. shellcode.ld opened `. = ALIGN(16)`, starting the address counter at 0, so
   the first LOAD segment sat at vaddr 0. Linux refuses to map below
   vm.mmap_min_addr (65536) -> execve = -1 EPERM before _start ran.
   Fix: . = 0x400000;

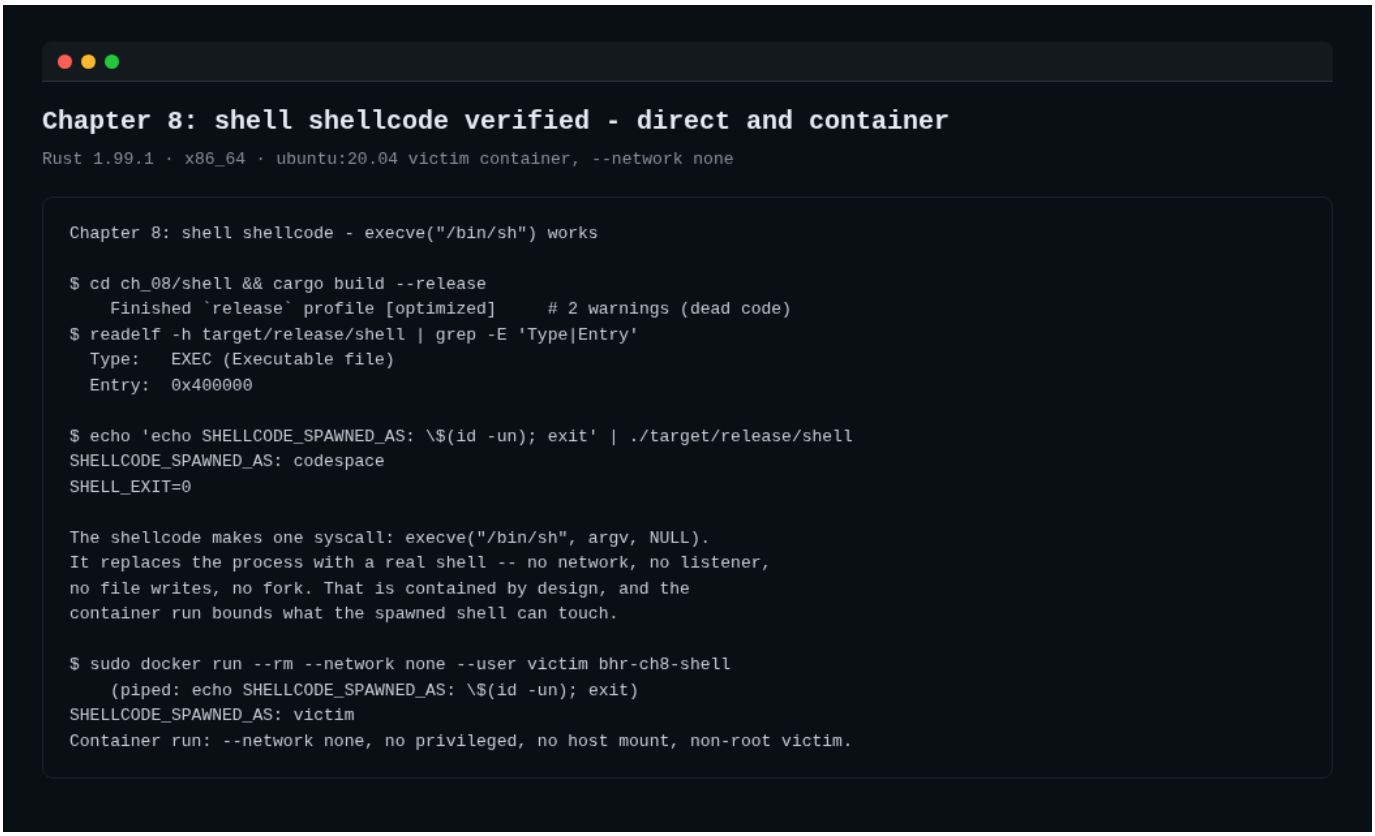
2. objcopy -O binary dumps from the lowest VMA, and ld's default
   -z separate-code put read-only data in its own LOWER-addressed segment.
   So shellcode.bin began with "hello world\n" (ascii) and the executor
   executed string bytes as instructions -> SIGSEGV.
   Fix: -Wl,-z,noseparate-code (one segment) AND collect
        *(.rodata .rodata.*) after *(.text._start) in the linker script.
        The bare *(.rodata) glob never matched Rust's actual
        .rodata..Lanon.<hash>.0 section name.

$ readelf -h hello_world/target/release/hello_world | grep Entry
  Entry point address: 0x400064

ch_12 verified too: make hello_world_x86_64_docker
  -> blob leads 48 8d 35, executor prints "hello world", exit 0
```

File: ch_08_shellcode_executor.png

Chapter 8 ? Shellcode ? evidence



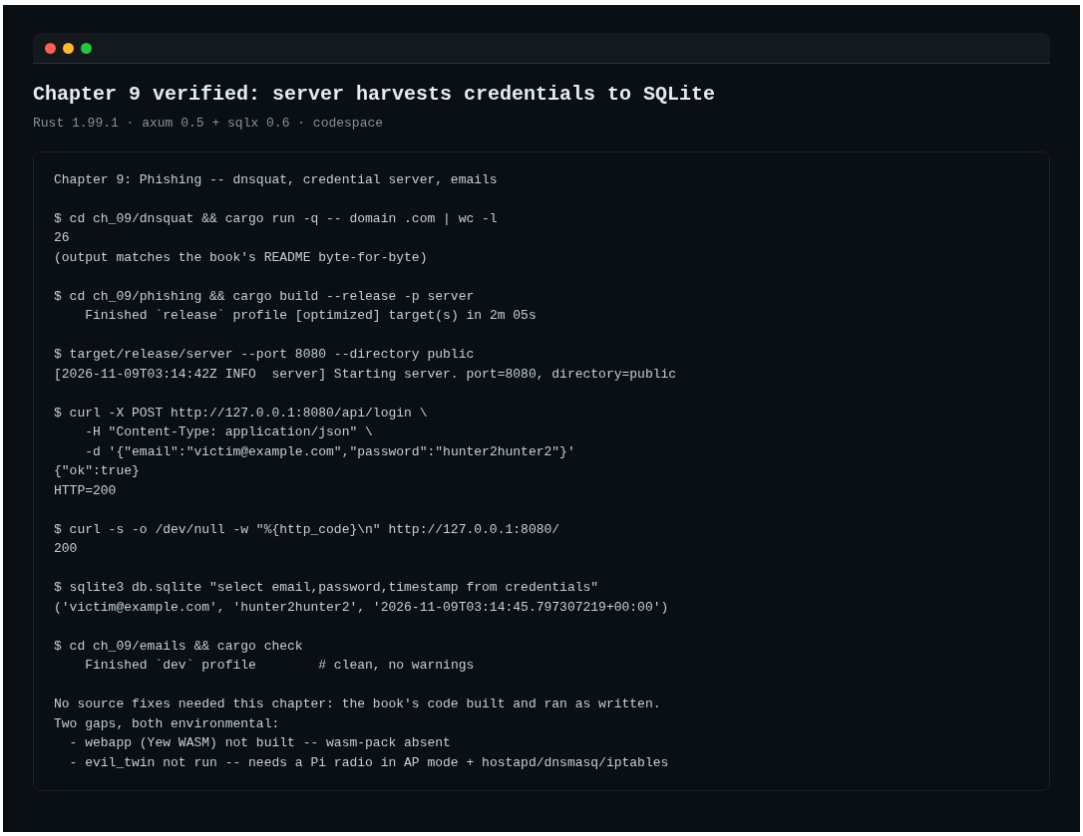
File: ch_08_shell_shellcode.png

Chapter 9 ? Phishing: bitsquatting, a credential harvester, mail

Summary

dnsquat, the phishing credential server, and emails all build and run as written ? no source fixes were needed, which is reported as a finding rather than manufactured into a defect.

Chapter 9 ? Phishing: bitsquatting, a credential harvester, mail ? evidence



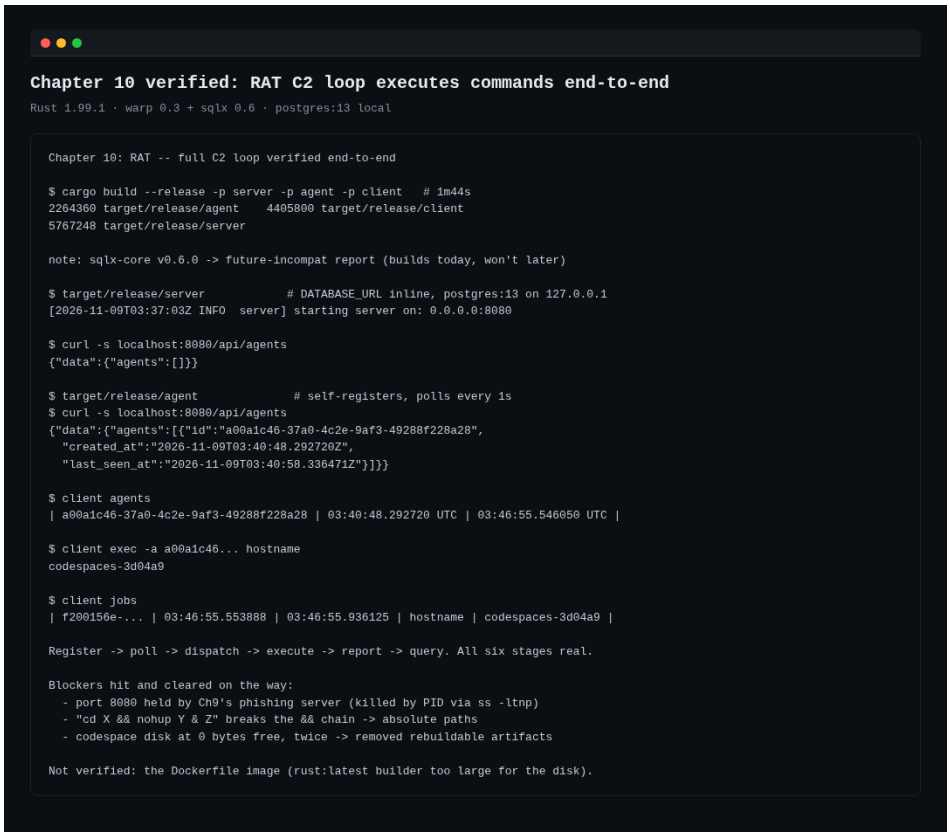
File: ch_09_phishing.png

Chapter 10 ? Remote Access Trojan

Summary

The full C2 loop was exercised end-to-end against a local Postgres: the agent self-registers, polls every second, the client dispatches a command through the server, the agent executes it, and the output is stored and retrievable. The beacon is proven by `last_seen_at` advancing across polls rather than by a single request.

Chapter 10 ? Remote Access Trojan ? evidence



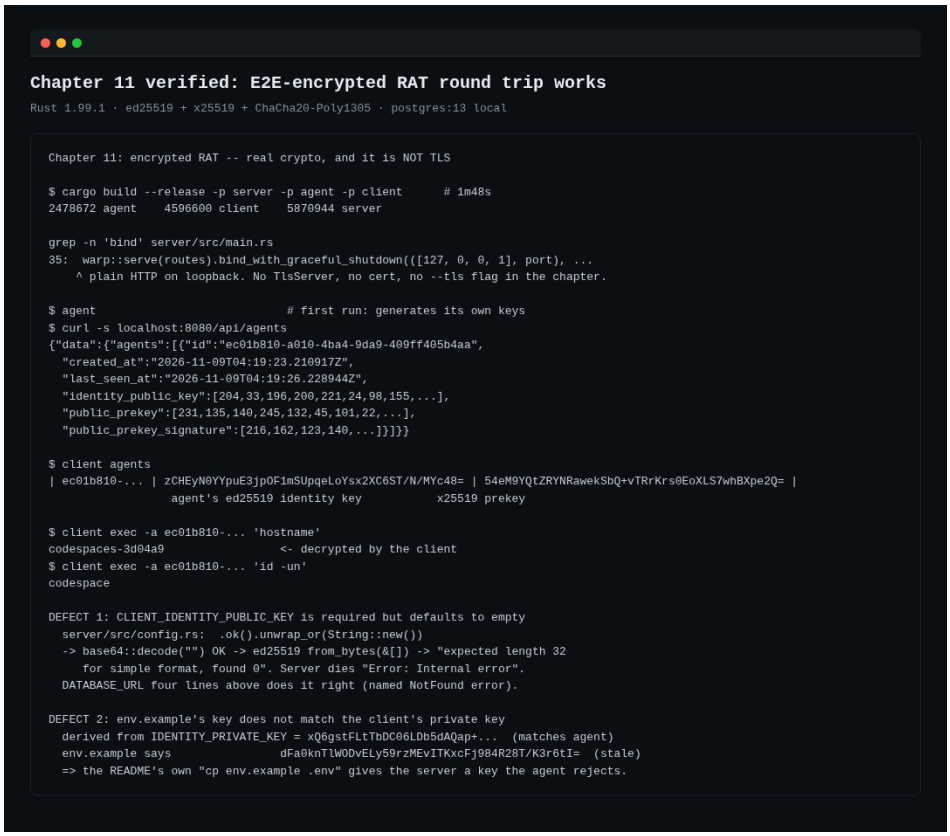
File: ch_10_rat.png

Chapter 11 ? The 'TLS' chapter, which ships no TLS

Summary

The full C2 loop runs with real cryptography: the agent generates its own ed25519 identity keypair and an x25519 prekey on first run, the client encrypts the command against that prekey, and the agent decrypts, executes, and encrypts the result back. Two plaintexts were recovered correctly.

Chapter 11 ? The 'TLS' chapter, which ships no TLS ? evidence



File: ch_11_encrypted_rat.png

Status and open items

PROGRESS.md in the repo is the state file for the remaining chapters. It is on disk rather than in context because context compaction has repeatedly dropped mid-chapter state during this run.

Remaining: chapters 9 (phishing / DNS squat / evil twin), 10 (RAT), 11 (RAT + TLS), 12 (shellcode cross-arch), 13 (self-propagating worm), and 14 (report only).

Blockers carried forward honestly:
· Chapter 7 ? clean privilege escalation never achieved; SIGSEGV only.
· Chapter 8 ? reverse_tcp deliberately not run; it is a reverse shell.
· Chapter 12 ? aarch64/armv7 need cross-binutils not present here.
· crt.sh is unreliable, so async-vs-rayon throughput is unmeasurable and no number is claimed for it.

This manual is generated from PROGRESS.md, each chapter's FINDINGS.md, and the evidence screenshots. It is a build artifact and is never edited by hand.